

16

Adding Authentication in Vue.js

In this chapter, you will learn how to use the Vue.js app's authentication process by building a registration form and login form. You will be creating a service for auth and router auth guards to protect any routers or pages in your Vue.js app. We will also be using a **JSON Web Token (JWT)** when requesting a protected resource. This chapter is an exciting chapter, so get let's started!

We will be covering the following topics:

- Setting up Vuex for authentication
- Writing an auth guard
- HTTP interceptors
- Auto login

Technical requirements

You will need Visual Studio Code to complete this chapter.

The following is a link to the GitHub repository for this chapter: <https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter16/>.

Setting up Vuex for authentication

The first section of this chapter is all about creating another module in our Vuex state management, which will be for our app's auth module. This section will also help you learn how to add a new module to our Vuex store. This module will require new action types, new actions, a new state, and new mutations.

Before we start implementing this new module for our auth, we will need help from the `jsonwebtoken` npm library. So, let's download the `jsonwebtoken` library.

Run the following npm command:

```
npm i jsonwebtoken
```

The preceding command will install the `jsonwebtoken` package in our application.

Then, create a new folder in the `src` directory and name it `auth`.

Now, create a JavaScript file called `auth.service.js` inside the `auth` directory. Add the following code to the `auth.service.js` file:

```
import api from "@/api/api-v1-config";

export async function loginUserAxios(login) {
  return await api.post("Users/auth", login);
}
```

In the preceding code, we created a service that takes a login object and sends a POST request to the `/api/v1/Users/auth` endpoint.

Next, we must create a folder in the `store` folder and name it `auth`.

Now, create a JavaScript file called `types.js` in the `store/auth` folder and add the following code:

```
export const LOGIN_USER = "LOGIN_USER";
```

The preceding code is an action type for logging in.

Next, we must create another JavaScript file called `actions.js` in the `store/auth` folder and add the following code:

```
import * as types from "../types";
import { loginUserAxios } from "@/auth/auth.service";

export async function loginUserAction({ commit }, payload) {
  try {
    const { data } = await loginUserAxios(payload);
    commit(types.LOGIN_USER, data.token);
  } catch (e) {}
}
```

Here, we imported the auth action types and `loginUserAxios` service and used them inside a newly created action called `loginUserAction`. The `loginUserAction` action requires a payload, and that payload will contain the username and email of the user trying to sign in.

Next, we must create another JavaScript file called `state.js` in the `store/auth` folder and add the following code to it:

```
const state = {
  signInState: {
    email: "",
    exp: Date.now(),
    sub: "",
    token: null,
  },
};
export default state;
```

The state of our auth module or namespace will have a `signInState` model with `email`, `exp`, `sub`, and `token` properties.

Now, create another JavaScript file called `mutations.js` in the `store/auth` folder and add the following code:

```
import * as types from "../types";
import * as jwt from "jsonwebtoken";

const mutations = {
  [types.LOGIN_USER](state, token) {
    state.signInState.token = token;
    const loginClaim = jwt.decode(token);
    claimToState(state, loginClaim);
    localStorage.setItem("token", token);
  },
};

export default mutations;

function claimToState(state, claim) {
  state.signInState.sub = claim.sub;
  state.signInState.email = claim.email;
  state.signInState.exp = claim.exp;
}
```

Here, we are importing the `types` and `jsonwebtoken` into the `mutations.js` file. We are then using the imported `types` and `jsonwebtoken` inside the mutation function. The added mutation takes a token, and then we store it in `signInState.token`. After this, we decode the token and store the decoded token in `loginClaim`.

You will notice that there's a function named `claimToState`. This function maps the claim's `sub`, `mail`, and `exp` to the `signInState`'s `sub`, `mail`, and `exp`.

After storing the `sub`, `email`, and `exp` properties of the claim in `signInState`, we must save the token in the browser's local storage. The keyname for the stored token value is `token`.

Next, we must create another JavaScript file called `getters.js` in the `store/auth` folder and then add the following code to it:

```
const getters = {
  email: (state) => {
    return state.signInState.email;
  },
}
```

```
isAuthenticated: (state) => {  
  return state.signInState.token;  
},  
};  
export default getters;
```

In the preceding code, we are creating a getter for `signInState.email`, and we are simply going to name it `email`.

Now, create an index file called `index.js` for the `store/auth` module:

```
import state from "../state";  
import getters from "../getters";  
import mutations from "../mutations";  
import * as actions from "../actions";  
export default {  
  namespaced: true,  
  getters,  
  mutations,  
  actions,  
  state,  
};
```

If the preceding code looks familiar to you, this is because we also wrote it in the `tour` module. Importing and exporting the state, getters, mutations, actions, and `namespaced` property set to `true` is what you will repeatedly be writing whenever you are creating a new module in the store.

The `namespaced` property set to `true` requires you to write the module's name whenever you are spreading `mapActions` or `mapGetters`. Take a look at the following code example, which is using a `foo` module:

```
...mapActions("fooModule", ["selectAction"]),
```

You will notice that the first argument is the namespace, and then the second argument is for the actions. The purpose of the namespace in the preceding code is to avoid name collisions or clashes if you have a large application and are using many modules that have the same names for actions.

Now, we must update the `index.js` file of the `store` folder:

```
import authModule from "../auth";
```

We are bringing `authModule` into the `index.js` file of the `store` and then using `authModule` in the `modules` object:

```
export default new Vuex.Store({  
  modules: {  
    tourModule,  
    authModule,  
  },  
  plugins,  
});
```

Now, `authModule` is integrated into our Vuex state management implementation.

Good! In the next section, we will learn how to protect pages of the Vue app.

Writing an auth guard

An **auth guard** is a piece of middleware in our router. We can write functions and trigger them in routes. An excellent example of a function that we can put in a router is a function that helps with authentication. The function would then check whether the user of the application is authenticated or not. We must then show specific pages to users that are authenticated. So, let's start writing our auth guard.

Create a JavaScript file called `auth.guard.js` in the `auth` folder and add the following code:

```
import store from "@/store";  
  
export const authGuard = (to, from, next) => {  
  console.log("authGuard");  
  const authRequired = to.matched.some((record) =>  
    record.meta.requiresAuth);  
  if (authRequired) {  
    if (store.getters["authModule/isAuthenticated"]) {  
      next();  
      return;  
    }  
  }  
}
```

```
    next("/login");  
  }  
  next();  
};
```

In the preceding code, we are defining our `authGuard`. Here, `authGuard` takes `to`, `from`, and `next` properties. We are also checking if a route has been tagged with `true`. We can do this by using the `authRequired` meta property from the route object.

If `auth` is required, we must check if the `isAuthenticated` getter returns `true`. If it does, we must go to the next pipe of the middleware. If it doesn't, we must go to the login page.

Now, let's update our router in the `router/index.js` file so that we can use the `authGuard` middleware we just wrote:

```
import { authGuard } from "@/auth/auth.guard";
```

Let's bring in our `authGuard`.

Now, write the following code and put it before the export default router line of code inside the `index.js` file of the router folder, like so:

```
router.beforeEach((to, from, next) => {  
  console.log("router.beforeEach");  
  authGuard(to, from, next);  
});  
export default router;
```

`router.beforeEach` is a global before guard and will trigger whenever we navigate inside our app. We included a `console.log` so that you can see that `beforeEach` runs every time we navigate.

Now, create a new Vue component called `Login.vue` inside the `auth/views` folder and add the following script:

```
<script>  
import { mapActions } from "vuex";  
import router from "@/router";  
export default {  
  name: "Login",  
  data: () => ({
```

```
    dialog: true,
    tab: null,
    login: {
      email: "",
      password: "",
    },
  }),
  methods: {
    ...mapActions("authModule", ["loginUserAction"]),
    onSubmit() {
      this.loginUserAction(this.login).then(() => {
        this.$router.push({ path: "/admin-dashboard" });
      });
    },
    navigateHome() {
      router.push("/");
    },
  },
};
</script>
```

Here, we are declaring some states in the script. We have a dialog, a tab, and a login object. We are then importing `mapActions` and `router` and using them in the `methods` block.

Next, we created two methods, namely `onSubmit` and `navigateHome`. `onSubmit` invokes `loginUserAction`, and if `loginUserAction` is successful, we must navigate to the `admin-dashboard` path. Meanwhile, `navigateHome` will simply send us to the home page.

Now, let's add the `template` syntax for the `Login.vue` component. Then, write the following code:

```
<template>
  <!-- https://vuetifyjs.com/en/styles/colors/#material-
  colors-->
  <v-app>
    <v-dialog
      v-model="dialog" persistent
```



```

max-width="600px" min-width="360px"
@click:outside="navigateHome"
>
<div>
  <v-tabs
    show-arrows background-color="pink accent-4"
    icons-and-text dark grow
  >
    <v-tabs-slider color="pink darken-4"></v-tabs-
      slider>
    <v-tab>
      <v-icon large>mdi-login</v-icon>
      <div>Login</div>
    </v-tab>
    <v-tab>
      <v-icon large>mdi-account-box-outline</v-icon>
      <div>Register</div>
    </v-tab>
    <!-- Login <v-tab-item> here -->
    <!-- Register <v-tab-item> here -->
  </v-tabs>
</div>
</v-dialog>
</v-app>
</template>

```

The template syntax here is a dialog component that renders two tabs – a tab for logging in and another tab for registering.

Now, let's replace the comment stating `<!-- Login <v-tab-item> here -->` with the following template syntax:

```

<v-tab-item>
  <v-card class="px-4">
    <v-card-text>
      <form @submit.prevent="onSubmit">
        <v-row>
          <v-col cols="12">

```

```
        <v-text-field label="E-mail" v-  
        model="login.email"></v-text-field>  
    </v-col>  
    <v-col cols="12">  
        <v-text-field  
            type="password" label="Password"  
            hint="At least 8 characters"  
            v-model="login.password" counter  
        ></v-text-field>  
    </v-col>  
    <v-col class="d-flex" cols="12" sm="6" xsm="12">  
    </v-col>  
    <v-spacer></v-spacer>  
    <v-col class="d-flex" cols="12" sm="3" xsm="12"  
    align-end>  
        <v-btn :disabled="false" color="primary"  
        type="submit"  
        >Login</v-btn>  
    </v-col>  
    </v-row>  
    </form>  
    </v-card-text>  
    </v-card>  
    </v-tab-item>
```

The preceding login tab provides a text field for the email and a text field for the password. The fields are wrapped with a form that triggers the `onSubmit` method.

Now, let's replace the comment stating `<!-- Register <v-tab-item> here -->` with the following template syntax:

```
<v-tab-item>  
    // ... please go to the repo for the complete code  
    <v-row>  
        <v-col cols="12" sm="6" md="6">  
            <v-text-field  
                label="First Name"  
                maxlength="20" required
```

```

        ></v-text-field>
    </v-col>
    <v-col cols="12" sm="6" md="6">
        <v-text-field
            label="Last Name"
            maxlength="20" required
        ></v-text-field>
    </v-col>
    <v-col cols="12">
        <v-text-field label="E-mail" required></v-text-
            field>
    </v-col>
    <v-col cols="12">
        <v-text-field
            counter label="Password"
            hint="At least 8 characters"
        ></v-text-field>
    </v-col>
    <v-col cols="12">
        <v-text-field
            block counter
            label="Confirm Password"
        ></v-text-field>
    </v-col>
    <v-spacer></v-spacer>
    <v-col class="d-flex ml-auto" cols="12" sm="3"
        xsm="12">
        <v-btn :disabled="false"
            color="primary">Register</v-btn>
    </v-col>
</v-row>
...
</v-tab-item>

```

The register tab is supposed to provide text fields for `firstName`, `lastName`, `password`, and `confirmPassword`. As you may recall, we don't have an API for creating a new user in ASP.NET Core. We hardcoded our user object in the `UserService.cs` file of our ASP.NET Core Web API project:

```
new User
{
    Id = 1,
    FirstName = "Yourname",
    LastName = "Yoursurname",
    Email = "yoursuperhero@gmail.com",
    Password = "Pass123!"
}
```

So, the login details we can use are the values of the email and password we provided in the preceding code.

Now, let's update the route and a meta information of the login path and the about path, like so:

```
{
    path: "/login",
    component: () => import("@/auth/views/Login"),
    meta: {
        requiresAuth: false,
    },
},
```

The preceding code adds the meta information to the route of login. Now, let's do the same thing to the route of about:

```
{
    path: "/about",
    name: "About",
    component: () => import("@/views/Main/About"),
    meta: {
        requiresAuth: false,
    },
},
```

The preceding code adds the meta information to the route of `about`. `requiresAuth` will be different for the path of `admin-dashboard`, though:

```
{
  path: "/admin-dashboard",
  component: () => import("@/views/AdminDashboard"),
  meta: {
    requiresAuth: true,
  },
}
```

Here, we are setting `requiresAuth` to `true`. This means that `admin-dashboard` and the rest of its pages can only be accessed by authenticated web users.

Next, we must update the `NavigationBar.vue` file of the `components` directory:

```
import { mapGetters } from "vuex";
```

Let's import `mapGetter` from `Vuex`.

Now, let's use `mapGetters` with the `authModule` key as our namespace in `mapGetters`, like so:

```
computed: {
  ...mapGetters("authModule", {
    isAuthenticated: "isAuthenticated",
    email: "email",
  }),
},
```

Now, we must map `isAuthenticated` and `email` to the local states that automatically get created for us.

Now, write a `v-btn` component from `Vuetify`, like so:

```
<v-btn
  v-if="isAuthenticated"
  color="primary"
  outlined
  :to="{ path: '/admin-dashboard' }"
>
  <span class="menu">Dashboard</span>
```

```

</v-btn>
<v-btn v-else color="primary" outlined :to="{ path:
'/login' }">
  <span class="menu">Login</span>
</v-btn>

```

The **Dashboard** menu button is only visible to authenticated users, while the **Login** menu button is visible to unauthenticated users.

Before we start sending requests to the backend, let's put the `[Authorize]` attribute back inside `ApiController.cs` with a namespace of `Travel.WebApi.Controllers.v1`:

```
[Authorize]
```

We are commenting out the `Authorize` attribute here so that the resources of the ASP.NET Core Web API are protected.

Let's rerun the server and select the **LOGIN** menu at the top of the screen:

The screenshot shows a web application's login interface. At the top, a magenta header bar contains two links: 'LOGIN' with a key icon and 'REGISTER' with a person icon. Below the header, the main content area has a light blue background. It features two input fields: 'E-mail' with the text 'yoursuperhero@gmail.com' and 'Password' with masked characters '.....'. A blue 'LOGIN' button is positioned at the bottom right of the form. A small number '8' is visible at the bottom right of the form area.

Figure 16.1 – Login form

The preceding screenshot shows the login form after clicking the **Login** menu. Use the following login credentials for authentication:

- **Email:** yoursuperhero@gmail.com
- **Password:** Pass123!

You should be able to log in, but an error alert prompt will appear. Open your browser's devtools and go to the **Network** | **Preview** tab:

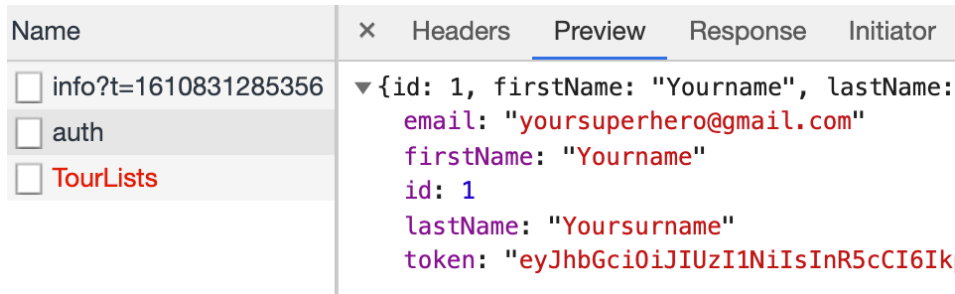


Figure 16.2 – Preview tab of the Network tab of DevTools

The preceding screenshot shows that the login was successful, but that the method that sends a GET request to fetch the tour lists collection failed. The fetch request failed because the ASP.NET Core Web API didn't find any JWT token in the header of our request. Hence, we are not authorized.

We will fix the JWT token in the header in the next section. But before that, let's check out the local storage of our browser to see if the Vue app can save the JWT token in local storage.

The successful login values, along with the token response, should be stored in **Local Storage**:

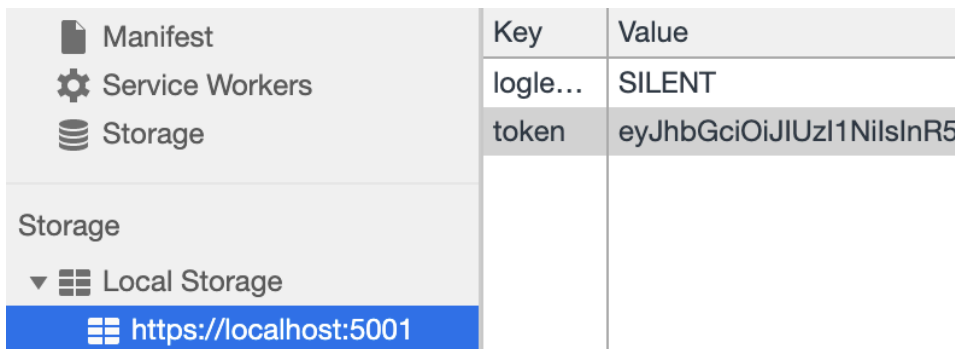


Figure 16.3 – Saved token in Local Storage

The preceding screenshot shows that the token is saved in the **Local Storage** area of the browser. We will use this token in our request to tell the ASP.NET Core Web API that we are authorized to read and write in the resource. We will do this in the next section.

HTTP interceptor

What is an HTTP interceptor? An HTTP interceptor is a piece of functionality that intercepts incoming HTTP responses and outgoing HTTP requests. We can automatically change or modify the header of our requests before sending them to the endpoints of web services. Luckily, Axios has HTTP interceptor interfaces readily available for developers to use. So, let's get started!

The first thing we must do is update the `auth.service.js` file of the `src/auth` folder with the following code:

```
const key = "token";
```

The token's key name is simply `key`.

We'll also be adding two functions to the `auth.service.js` file:

```
export function getToken() {  
  return localStorage.getItem(key);  
}  
  
export function logout() {  
  localStorage.clear();  
  window.location = "/login";  
}
```

The `getToken` function gets the token's value from the local storage, while the `logout` function clears out any stored values in the local storage and then redirects the users to the login page.

Now create a new JavaScript file called `interceptors.js` in the `api` folder and write the following code in it:

```
import { getToken } from "@/auth/auth.service";  
  
export function interceptorsInit(axiosInstance) {  
  axiosInstance.interceptors.request.use(function(options) {  
    const jwtToken = getToken();
```



```

    if (jwtToken) {
      options.headers["Authorization"] = `Bearer ${jwtToken}`;
    }
    return options;
  });

  axiosInstance.interceptors.response.use(
    (response) => response,
    (error) =>
      Promise.reject(
        (error.response && error.response.data) || "Something
        went wrong"
      )
  );

  return axiosInstance;
}

```

Here, we are bringing in the `getToken` service and creating a function called `interceptorsInit`. The function takes an instance of `Axios` and then uses it to trigger `getToken`. If the token is present in local storage, we must add an authorization key to the header with the Bearer space token value. Pay attention that the value of the token is an interpolation of the Bearer string in the token.

We are also adding an interceptor for any response. You can add any modification to the response if you wish.

Now, let's update the `api-v1-config.js` file of the `api` folder by importing `interceptorsInit`:

```
import { interceptorsInit } from "@api/interceptors";
```

We can now use `interceptorsInit`. Let's use `interceptorsInit` by passing the instance of `Axios`, like so:

```
api = interceptorsInit(api);
```

The return value of `interceptorsInit` is then stored in the same instance, which modifies `Axios`:

Now rerun the application. Log out and log in again to see if we can fetch the Tour Lists right after logging in.

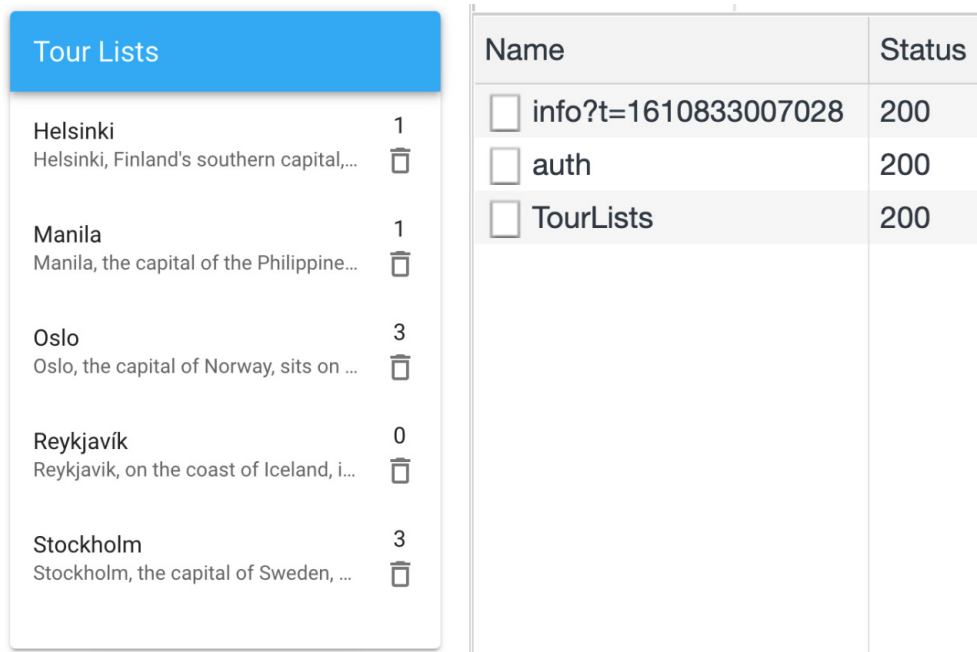


Figure 16.4 – Tour Lists status code 200

The preceding screenshot shows that the `getTourListsAxios` service can get the Tour List without any errors. Here, you can see that the response code is **200** for the **auth** endpoint and **200** for the **TourLists** endpoint.

Now, let's try to log out and see if the token will be removed from the browser.

Open the dev tools of your browser and go to the application tab, like so:

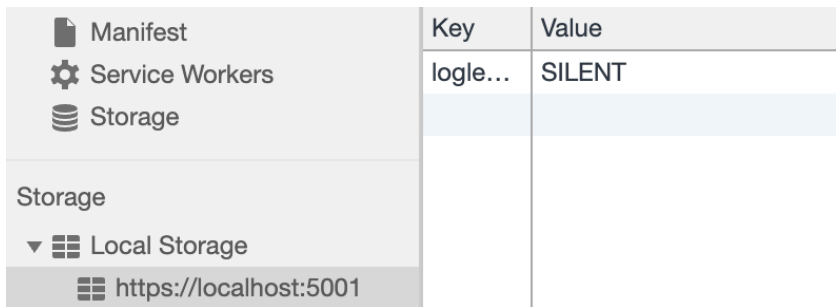


Figure 16.5 – Empty local storage

The preceding screenshot shows that logging out removes the token from the browser.

At this point, we know that authentication using Vuex is working, the auth guard is working, and that the HTTP interceptor of the app is also working. Now, what if the user closes the browser and then returns to the web app at a later date?

Here, we must create a mechanism where logging in is automatic if a token in local storage is present and valid. We will build this feature, where the user doesn't have to enter credentials, in the next section.

Auto login

To give our web users a user-friendly web app when logging in, we will create an auto login. The idea is that whenever a token is present in local storage, we will check if it's a valid token and then use it for authentication. By doing this, our web visitors don't need to reenter their login details, which is sometimes annoying for them.

To start, let's update the `auth.service.js` file of the `src/auth` folder and then import `jsonwebtoken`, like so:

```
import * as jwt from "jsonwebtoken";
```

We will use the JWT token later to decode the token.

Now, add two new functions to the `auth.service.js` file:

```
export function isTokenFromLocalStorageValid() {  
  const token = localStorage.getItem(key);  
  if (!token) {  
    return false;  
  }  
  
  const decoded = jwt.decode(token);  
  const expiresAt = decoded.exp * 1000;  
  const dateNow = Date.now();  
  return dateNow <= expiresAt;  
}  
  
export function getUserEmailFromToken() {  
  const token = localStorage.getItem(key);  
  if (!token) return false;  
  const decoded = jwt.decode(token);
```

```
    return decoded.email;
  }
}
```

`isTokenFromLocalStorageValid` gets the token, decodes it, and then checks its expiration, while `getUserEmailFromToken` returns the email after getting the token from local storage and decoding it.

Let's update the route of the app:

```
import { isTokenFromLocalStorageValid } from "@/auth/auth.service";
```

Here, we are importing the `isTokenFromLocalStorageValid` service.

Now, we are going to add a guard hook called `beforeEnter` to the login route. `beforeEnter` guards the route where we put it. We are going to run some logic before the app reads the login route object. Now, update the login route object, like so:

```
{
  path: "/login",
  component: () => import("@/auth/views/Login"),
  meta: {
    requiresAuth: false,
  },
  beforeEnter: (to, from, next) => {
    const valid = isTokenFromLocalStorageValid();
    console.log("VALID::", valid);
    if (valid) {
      next("/continue-as");
    } else {
      next();
    }
  },
},
{
  path: "/continue-as",
  component: () => import("@/auth/views/ContinueAs"),
  meta: {
```

```

      requiresAuth: false,
    },
  },
},

```

Here, we are adding a new page called `Continue As` to our routes. We will use the `Continue As` page to give our web visitors the option to use the account from the token or to log out. We will then use the `Continue As` page to reroute a user if the token in local storage is no longer valid.

Now, we will save a decoded token in our global state if the token is still valid. Let's update the `types.js` file of the `store/auth` folder and add the following code to it:

```

export const LOCAL_STORAGE_TOKEN_LOG_IN = "LOCAL_STORAGE_TOKEN_LOG_IN";

```

The preceding code is a new action type we are adding to the file. Now, let's update the `actions.js` file of the `store/auth` folder and add the following code to it:

```

import {
  loginUserAxios,
  isTokenFromLocalStorageValid,
  getToken
} from "@/auth/auth.service";

```

Here, we are bringing in the `isTokenFromLocalStorageValid` service and the `getToken` service.

Now, let's create a new Vuex action for our auth module:

```

export function useLocalStorageTokenToSignInAction({ commit }) {
  {
    if (!isTokenFromLocalStorageValid()) {
      return;
    }
    const token = getToken();
    commit(types.LOCAL_STORAGE_TOKEN_LOG_IN, token);
  }
}

```

This new action checks the validity of the token and then commits the token using the `LOCAL_STORAGE_TOKEN_LOG_IN` action type.

Now, let's update the `mutations.js` file of the `store/auth` folder:

```
[types.LOCAL_STORAGE_TOKEN_LOG_IN](state, token) {  
  state.signInState.token = token;  
  const loginClaim = jwt.decode(token);  
  claimToState(state, loginClaim);  
},
```

The new mutation stores the token in `signInState.token`, decodes the token, and then maps the information of the decoded token to the other properties of `signInState`.

Next, we must update the `auth.guard.js` file of the `src/auth` folder with the following code:

```
import { isTokenFromLocalStorageValid } from "../auth.service";  
  
if (authRequired) {  
  if (store.getters["authModule/isAuthenticated"]) {  
    next();  
    return;  
  } else if (isTokenFromLocalStorageValid()) {  
    next();  
    return;  
  }  
  next("/login");  
}
```

Here, we are bringing in the `isTokenFromLocalStorageValid` service and using it to validate a token inside `authGuard`.

Now, we can create the `ContinueAs.vue` component, which is the page we recently added to our route. Write the following script for the `ContinueAs` page:

```
<script>  
import { getUserEmailFromToken, logout } from "@/auth/auth.  
service";  
export default {  
  name: "ContinueAs",  
  data: () => ({
```

```

    email: getUserEmailFromToken(),
  }},
  methods: {
    handleLogout() {
      logout();
    },
    onSubmit() {
      this.$router.push({ path: "/admin-dashboard" });
    },
  },
};
</script>

```

The `ContinueAs` page has an `email` state that gets its value from the `getUserEmailFromToken` function. The `ContinueAs` page also has a `method` and `onSubmit`. The `handleLogout` method logs the user out, while the `onSubmit` method redirects the user to the Admin Dashboard page.

Now, for the template syntax of the `ContinueAs` page, copy the following code:

```

<template>
  <div class="container" v-if="email">
    <div class="text-h4 my-5">Do you like to continue as {{
      email }}? </div>
    <v-btn @click="onSubmit" color="primary" class="mr-4">Yes</v-btn>
    <v-btn @click="handleLogout()" outlined color="primary">
      No thanks, I'd like to log out</v-btn>
    </div>
    <div v-else class="text-h4 m-10">
      Fancy meeting you here.
      <v-btn @click="handleLogout()" outlined color="primary">
        >Go to Home Page</v-btn>
    </div>
  </template>

```

This template is a simple UI that gives our web users the option to proceed with using their accounts or not.

Now, let's update the `index.vue` file of the `views/AdminDashboard` folder with the following code for the `script` section:

```
import { mapActions, mapGetters } from "vuex";
```

Here, we are importing `mapActions` and `mapGetters` again. Next, we must create a method called `localStorageLogin` that will invoke `useLocalStorageTokenToSignInAction`:

```
methods: {  
  ...mapActions("authModule",  
    ["useLocalStorageTokenToSignInAction"]),  
  
  localStorageLogin() {  
    this.useLocalStorageTokenToSignInAction().then();  
  }  
},  
  
computed: {  
  ...mapGetters("authModule", {  
    email: "email",  
  })  
},  
  
mounted() {  
  this.localStorageLogin();  
},
```

We are also bringing in the `email` getter from our `auth` module. Let's replace the hardcoded text `email` here with a state from our store:

```
<v-list-item-content>  
  <v-list-item-subtitle>{{ email }}</v-list-item-subtitle>  
</v-list-item-content>
```

The sidebar's email will now become dynamic based on the email address of the user.

Now, rerun the app, log in, and close the browser. Revisit the Vue app's URL.

You should see a question in heading format stating *Do you like to continue as*, like so:

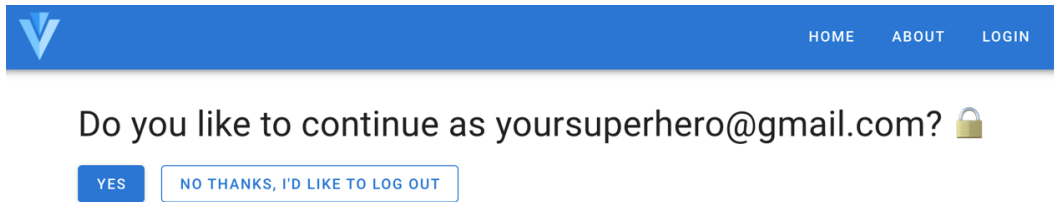


Figure 16.6 – Auto login with the option to proceed or not

The preceding screenshot shows the `ContinueAs` page, along with a question and the user's email address. The Vue app redirected you to the `ContinueAs` page since your browser contains the stored token that it got from the ASP.NET Core Web API response when you logged in.

Click **YES** to proceed to the Admin **Dashboard**:

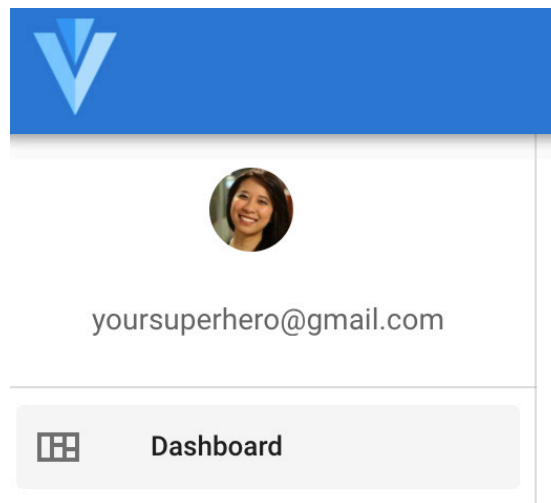


Figure 16.7 – Email address on the sidebar

The preceding screenshot shows the **yoursuperhero@gmail.com** email address on the sidebar, which means that the token's details have been saved in the global state store.

With that, we have finished protecting our Vue.js app by implementing a few techniques. There are still some missing parameters that we can use to harden the protection of our app.

If you are building a complex Vue app and only have a little time to finish your project, consider using an Identity as a Service to authenticate your users. There are several options for this, such as AWS Incognito, Azure AD B2C, CGP Identity Platform, Okta, and Auth0, which I've used in several of my projects because of their superb authentication SDK and the amount of documentation they have. The Identity as a Services I've mentioned will significantly speed up your development, as well help you identify the security measures you will need in your application.

So, let's wrap things up and see what you have learned in this chapter.

Summary

Let's quickly summarize what you have learned in this chapter. First, you learned how to set up a Vuex store for authentication and practiced how to set up a module in the store, end to end. You then learned about auth guard, which protects the route. You also learned how to intercept outgoing HTTP requests, which helps automatically modify every header of your request. Lastly, you learned how to create a simple auto login for your users to improve their user experience.

In the next chapter, we will create some validations in our forms to give our users a better user experience when they're filling out forms.